IoT Report

Identification and analysis of a chosen sample of malware

Samuel Rutherford

CMP408: Advanced Ethical hacking

BSc Ethical Hacking Year 4

2022/23

Note that Information contained in this document is for educational purposes.

Contents

1 INTRODUCTION

1.1 BACKGROUND

The development of malicious software used in red team operations can greatly help to develop an understanding of they operate and create suitable countermeasures. Keyloggers are commonly used by hackers as a method of acquiring sensitive data from a victim such as passwords and bank details.

Relevance to IoT and cloud security

In order to effectively and securely perform operations on an IoT device, considerations with how a program interacts with the kernel space and user space must be taken seriously. Loadable Kernel Modules (LKM) are critical to correctly utilising the kernel space (or ring0) where interactions with physical input and output devices are critical to the programs purpose, such as how GPIO pins on a raspberry pi interact with a button. LKMs allow for communication between the kernel space and the user space such as passing information of certain devices. An issue with LKMs is that they can pose a serious security risk due to how closely they operate with the Linux kernel. If a hacker was able to make an unsuspecting victim use their LKM, the hacker could create a backdoor, perform privilege escalation and more.

Using AWS (Amazon Web Services) is an extremely effective method of using cloud computing. Due to the flexibility it allows, the security of the instances, services and applications the user is running is a shared responsibility between themselves and AWS. Data leakage is a possible issue which can be alleviated through the use of HTTPS/TLS to provide end to end data encryption.

1.2 OBJECTIVES

Within this keylogger project there are numerous objectives covering the different aspects of hardware, software and cloud computing. Additionally, implementing secure development practices is critical to the project. These objectives include:

- Development of an LKM to receive a button press.
- Create a userspace program to receive and read the button press from the LKM.
- Utilise libraries to perform the keylogging.
- Have an LED light up when the PI is performing keylogging.
- Process the keylogs and send them to a database.
- Display the information on a webapp.

2 METHODOLOGY

2.1 RASPBERRY PI

The development of the project began by focusing on the raspberry pi. This was done first to complete both the hardware and software components of the project at the same time. The development of the code was primarily performed on a Linux virtual machine and on the pi itself.

2.1.1 Button functionality

The goal for the button is to once pressed, start the keylogger function. This is achieved by connecting a physical button to a GPIO pin on the raspberry pi. On the breadboard, the button was connected to GPIO pin 2 along with a resistor to a ground pin. The code for the LKM to read the button input was created using C and involved reading a file called 'button_device'. The program begins with an initialisation function to create the 'button_device' device file, validating the GPIO pin and further error checking. Depending on if the button was pressed or released, the LKM would communicate that information with the userspace program. This is achieved using various ioctl (input output control) functions. Ioctl involves identifying the devices specific files and identifying special parameters. The ioctl functions found within the LKM are either for registering the PID (process id) for the userspace program to enable signal communication which is found in IOCTL_REGISTER_APP, or for actually reading the button state with IOCTL_READ_BUTTON_STATE:

```
switch (cmd) {
    case IOCTL_READ_BUTTON_STATE:
        // Copy button_state to userspace
        retval = copy_to_user(user_arg, &button_state, sizeof(button_state));
        if (retval != 0) {
            printk(KERN_ALERT "Failed to copy data to user space\n");
            return -EFAULT;
        }
        printk(KERN_INFO "IOCTL read. Current state: %d\n", button_state);
        break;

    case IOCTL_REGISTER_APP:
        if (copy_from_user(&registered_pid, user_arg, sizeof(registered_pid))) {
            printk(KERN_ALERT "Failed to copy PID from user space\n");
            return -EFAULT;
        }
        printk(KERN_INFO "User-space app registered with PID: %d\n", registered_pid);
        break;
```

Figure 2-

Once there has been a successful button press, the LKM communicates with userspace program using signals.

2.1.2 Userspace button reading

In order to run the keylogger in the userspace, the userspace program known as 'ButtonIOCTL4.c' runs and uses a while loop to continuously read the button_state and any signals the LKM may send. To establish this connection, the PID of the userspace program is sent to the LKM to register the process.

Upon receiving a signal using the signal.h library or reading the button_state as 1, the program will run the keylogging function known as runLogger().

2.1.3 Key logging

Within logger4.c, a single function exists to record the keyboard inputs from the user. This is achieved using the library known as evtest which directly reads from the devices event file. The keyboard being logged must have its event file identified manually which can be achieved by navigating to /dev/input/ and viewing the eventX file.

[insert image of event0 file]

The output of evtest is then saved to a text document to be later processed and sent to the cloud. Additionally, when the function is activated, an LED is turned on to indicate to the user that it is currently performing keylogging.

2.1.4 Pi to AWS communication

To send the keylogging text document to the AWS services, the pi uses a python program to accomplish this. The python program uses the library known as boto3 to communicate with the AWS services. The program requires the users AWS access keys, session token and region. Additionally, a lambda client is created which sends the keylogging text file to the lambda function known as KeyloggerFunc which is hosted by AWS.